

LINEAR DATA STRUCTURES BASICS

Lists

Author: Eng. Carlos Andrés Sierra, M.Sc.
cavirguezs@udistrital.edu.co

Full-time Adjunct Professor
Computer Engineering Program
School of Engineering
Universidad Distrital Francisco José de Caldas

2026-I



Course Outline



ADT Concept and Definition

Definition

An **Abstract Data Type (ADT)** is a **mathematical model** that defines **data** and **operations** without specifying how they are implemented. ①

Key Characteristics of ADTs:

- ② Primitive (native)

integer
decimal
boolean
characters

create our own datatype



ADT Concept and Definition

Definition

An **Abstract Data Type (ADT)** is a mathematical model that defines data and operations without specifying how they are implemented.

Key Characteristics of ADTs:

- **Data Abstraction:** Focus on what operations do, not how
- **Encapsulation:** Hide implementation details *get/set*
- **Interface:** Define clear methods for interaction
- **Independence:** Implementation can change without affecting users



ADT Operations

Example

List ADT Operations:

- Add element to list
- Remove element from list
- Find element in list
- Get size of list
- Check if list is empty

encapsulation



Interface vs. Implementation Separation I

abstract

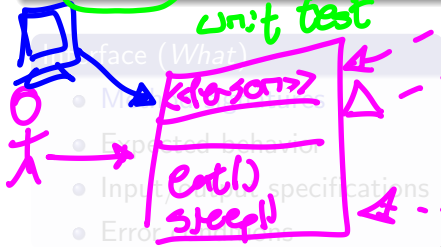
concrete

Why Separate Interface from Implementation?

1. **Flexibility:** Can change how things work internally
2. **Maintainability:** Easier to fix and improve code
3. **Reusability:** Same interface, different implementations
- **Testing:** Can test interface independently

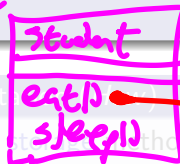


unit test



Implement

- Data structure method
- Algorithm details
- Memory management
- Performance optimizations



maintain



Interface vs. Implementation Separation I

Why Separate *Interface* from *Implementation*?

- **Flexibility:** Can change how things work *internally*
- **Maintainability:** Easier to *fix* and improve code
- **Reusability:** Same interface, *different* implementations
- **Testing:** Can test interface *independently*

Interface (*What*)

- Method signatures
- Expected behavior
- Input/output specifications
- Error conditions

documentation

abstract
abstract

output
type return
name (params);
name2 (params);
Input



Interface vs. Implementation Separation I

Why Separate *Interface* from *Implementation*?

- **Flexibility:** Can change how things work *internally*
- **Maintainability:** Easier to *fix and improve* code
- **Reusability:** Same interface, *different* implementations
- **Testing:** Can test interface *independently*

Interface (*What*)

- Method signatures
- Expected behavior
- Input/output specifications
- Error conditions

Implementation (*How*)

- Data storage method
- Algorithm details
- Memory management
- Performance optimizations

attributes → *logic*

HEAP SIZE →

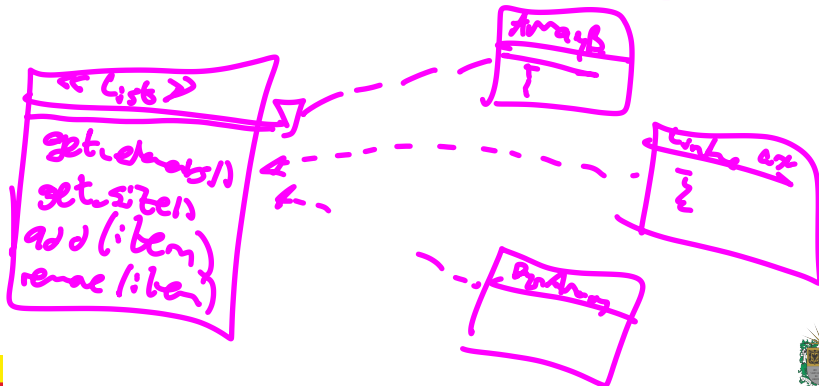


Interface vs. Implementation Separation II

Example

List Interface: `add(item)`, `remove(item)`, `size()`

Implementations: Array-based, Linked-list-based, Dynamic array



Study Case: Student Enrollment System Using Lists I

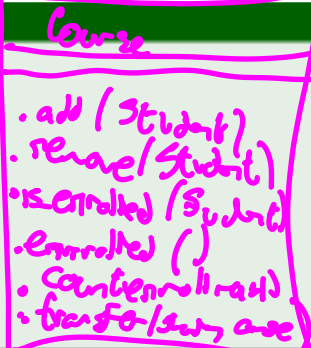
Problem

A university needs a system to manage **student enrollment** in courses. Students can be **added**, **removed**, and **searched** in course rosters.

Example

System Requirements:

- **Add** student to course
- **Remove** student from course
- **Check** if student is enrolled
- **List** all enrolled students
- Get **enrollment count**
- **Transfer** student between courses



Course

- *add (Student)*
- *remove (Student)*
- *isEnrolled (Student)*
- *enrolled ()*
- *countEnrollment*
- *transfer / swap*

Study Case: Student Enrollment System Using Lists II

not-prim-
value assign
 $a = 5 \rightarrow 0x30$
 $b = a \rightarrow 0x21$

A possible solution:

Grake → make

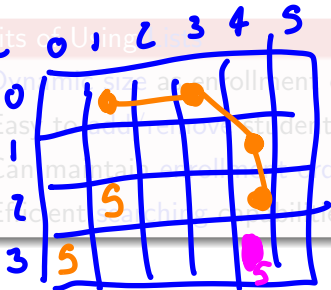
EnrollmentSystem

- + addStudent(courseId, student)
- + removeStudent(courseId, student)
- + isEnrolled(courseId, student)
- + getEnrollmentCount(courseId)
- + listStudents(courseId)

— class
} methods

Benefits of Using Lists

- Dynamic size as enrollment changes
- Easy to add/remove students
- Can maintain enrollment order
- Efficient searching capabilities



$x = 5$

$x \rightarrow 0x34$
Pointer

$obj = Student()$

$obj \rightarrow 0x01$ (4 slots)

$y = obj$

No pointers



Study Case: Student Enrollment System Using Lists II

EnrollmentSystem

- + addStudent(courseId, student)
- + removeStudent(courseId, student)
- + isEnrolled(courseId, student)
- + getEnrollmentCount(courseId)
- + listStudents(courseId)

A possible solution:

Benefits of Using Lists

- **Dynamic size** as enrollment changes
- Easy to **add/remove** students
- Can maintain **enrollment order**
- **Efficient searching** capabilities



Static vs. Dynamic Lists I

Static Lists

Characteristics:

- Fixed size at creation
- Uses arrays as base structure
- Memory allocated at compile time
- Fast access by index

Dynamic Lists

Characteristics:

- Size changes during execution
- Uses pointers or expandable arrays
- Memory allocated at runtime
- Flexible size management

When to Use Each

- Static known size, frequent access, simple operations
- Dynamic unknown size, frequent insertions/deletions



Static vs. Dynamic Lists I

Static Lists

Characteristics:

- Fixed size at creation
- Uses arrays as base structure
- Memory allocated at compile time
- Fast access by index

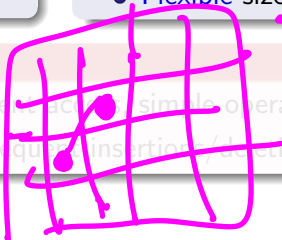
Dynamic Lists

Characteristics:

- Size changes during execution
- Uses pointers or expandable arrays
- Memory allocated at runtime
- Flexible size management

When to Use Each

- Static: Known size, frequent access, simple operations
- Dynamic: Unknown size, frequent insertion/deletion



Static vs. Dynamic Lists I

Static Lists

Characteristics:

- Fixed size at creation
- Uses arrays as base structure
- Memory allocated at compile time
- Fast access by index

Dynamic Lists

Characteristics:

- Size changes during execution
- Uses pointers or expandable arrays
- Memory allocated at runtime
- Flexible size management

When to Use Each

search

- Static: Known size, frequent access, simple operations
- Dynamic: Unknown size, frequent insertions/deletions



Static vs. Dynamic Lists II

Static Lists

Advantages:

- Simple implementation
- Fast random access
- No memory overhead

Disadvantages:

- Fixed maximum size
- Memory waste if not full
- Insertion/deletion expensive

Dynamic Lists

Advantages:

- No size limitations
- Efficient memory usage
- Easy insertion/deletion

Disadvantages:

- More complex implementation
- Memory overhead for pointers
- Slower access in some cases



Static vs. Dynamic Lists II

Static Lists

Advantages:

- Simple implementation
- Fast random access
- No memory overhead

Disadvantages:

- Fixed maximum size
- Memory waste if not full
- Insertion/deletion expensive

Dynamic Lists

Advantages:

- No size limitations
- Efficient memory usage
- Easy insertion/deletion

Disadvantages:

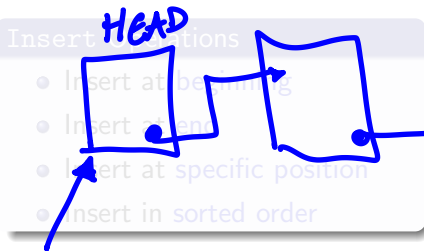
- More complex implementation
- Memory overhead for pointers
- Slower access in some cases



List Operations I: Insert, Search

Core List Operations

All list implementations must support fundamental operations for data manipulation.



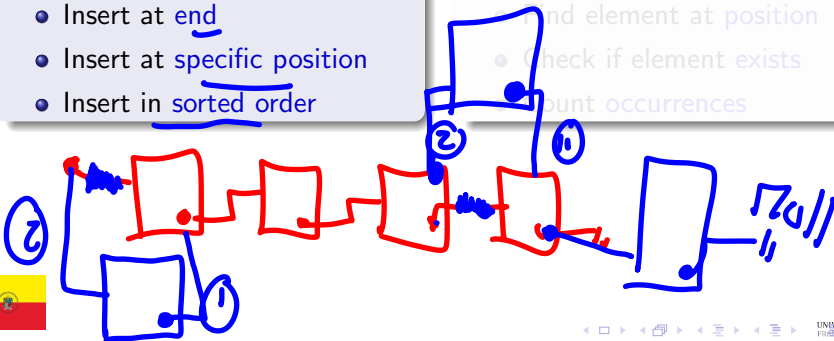
List



All list implementations must support **fundamental operations** for data manipulation.

- Insert at beginning
- Insert at end
- Insert at specific position
- Insert in sorted order

- Find element by **value**
- Find element at **position**
- Check if element **exists**
- Count **occurrences**



List Operations I: Insert, Search

Core List Operations

All list implementations must support **fundamental operations** for data manipulation.

Insert Operations

- Insert at **beginning**
- Insert at **end**
- Insert at **specific position**
- Insert in **sorted order**

Search Operations

- Find element by **value**
- Find element at **position**
- Check if element **exists**
- Count **occurrences**

→ loop → pointers



List Operations II: Delete, Update

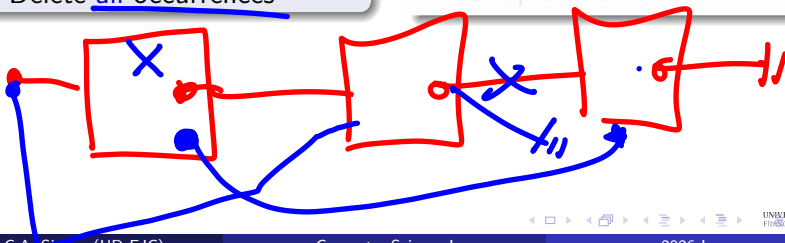
Core List Operations

All list implementations must support **fundamental operations** for data manipulation.

Delete Operations

- Delete by value
- Delete at position
- Delete first occurrence
- Delete all occurrences

- 1) begin ✓
 - 2) end → pre-end
 - 3) item
- Swap elements



List Operations II: Delete, Update

Core List Operations

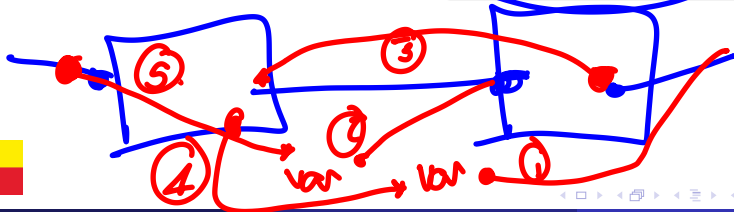
All list implementations must support **fundamental operations** for data manipulation.

Delete Operations

- Delete by **value**
- Delete at **position**
- Delete **first** occurrence
- Delete **all** occurrences

Update Operations

- Update element at **position**
- Update **first** occurrence
- Update **all** occurrences
- **Swap** elements



Study Case: Shopping Cart Implementation with *Array-based List*

Problem

Implement a **shopping cart** system where customers can **add** items, **remove** items, **update quantities**, and view their cart contents.

ShoppingCart
- items: Array[maxSize] - itemCount: Integer
+ addItem(product, quantity, price) + removeItem(productName) + updateQuantity(productName, newQuantity) + getTotalPrice() + getItemCount() + displayCart()



Study Case: Shopping Cart Implementation with Array-based List I

Problem

Implement a **shopping cart** system where customers can **add** items, **remove** items, **update quantities**, and view their cart contents.

ShoppingCart
- items: Array[maxSize] - itemCount: Integer
+ addItem(product, quantity, price) + removeItem(productName) + updateQuantity(productName, newQuantity) + getTotalPrice() + getItemCount() + displayCart()



Study Case: Shopping Cart Analysis with *Array-based List II*

Array Implementation Benefits

- Fast access to any item by index
- Simple memory layout
- Good for small, bounded carts
- Easy to implement sorting (by price, name)

Considerations

- Need to handle cart size limit
- Shifting elements when inserting/deleting
- Memory allocated upfront
- Good for typical shopping scenarios (few items)



Study Case: Shopping Cart Analysis with *Array-based List II*

Array Implementation Benefits

- Fast access to any item by index
- Simple memory layout
- Good for small, bounded carts
- Easy to implement sorting (by price, name)

Considerations

- Need to handle cart size limit
- Shifting elements when inserting/deleting
- Memory allocated upfront
- Good for typical shopping scenarios (few items)



Dynamic Memory Allocation I

Definition

Dynamic memory allocation allows programs to **request memory during execution**, enabling data structures to **grow and shrink** as needed.

Key Concepts

- **Runtime Allocation:** Memory requested when needed
- **Heap Memory:** Memory pool for dynamic allocation
- **Pointers:** Variables that store memory addresses
- **Memory Management:** Allocation and deallocation



Dynamic Memory Allocation I

Definition

Dynamic memory allocation allows programs to **request memory during execution**, enabling data structures to **grow and shrink** as needed.

Key Concepts

- **Runtime Allocation:** Memory requested **when needed**
- **Heap Memory:** Memory **pool** for dynamic allocation
- **Pointers:** Variables that store **memory addresses**
- **Memory Management:** Allocation and deallocation



Dynamic Memory Allocation II

Advantages

- Flexible memory usage
- No size limits at compile time
- Memory used only when needed
- Can handle varying data sizes

Challenges

- Memory leaks possible
- Fragmentation issues
- Slower than static allocation
- More complex programming

Best Practices

- Always free allocated memory
- Check for allocation failures
- Avoid memory leaks
- Use appropriate allocation strategies

Dynamic Memory Allocation II

Advantages

- Flexible memory usage
- No size limits at compile time
- Memory used only when needed
- Can handle varying data sizes

Challenges

- Memory leaks possible
- Fragmentation issues
- Slower than static allocation
- More complex programming

Best Practices

- Always free allocated memory
- Check for allocation failures
- Avoid memory leaks
- Use appropriate allocation strategies

Dynamic Memory Allocation II

Advantages

- Flexible memory usage
- No size limits at compile time
- Memory used only when needed
- Can handle varying data sizes

Challenges

- Memory leaks possible
- Fragmentation issues
- Slower than static allocation
- More complex programming

Best Practices

- Always free allocated memory
- Check for allocation failures
- Avoid memory leaks
- Use appropriate allocation strategies

Pointer-based List Structures I

Definition

Pointer-based lists use **dynamic memory allocation** and **pointers** to create flexible data structures where elements can be stored **anywhere in memory**.

Node Structure

Each element (**node**) contains:

- **Data:** The actual **information** stored
- **Pointer:** Reference to the next element
- **Optional:** Additional pointers for **complex structures**



Pointer-based List Structures I

Definition

Pointer-based lists use dynamic memory allocation and pointers to create flexible data structures where elements can be stored anywhere in memory.

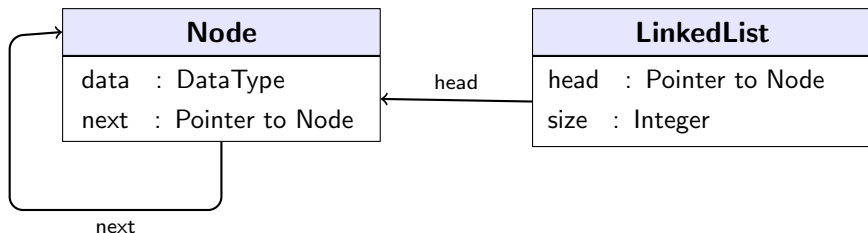
Node Structure

Each element (**node**) contains:

- **Data:** The actual information stored
- **Pointer:** Reference to the next element
- **Optional:** Additional pointers for complex structures



Pointer-based List Structures II



Pointer-based List Structures III

Advantages

- Dynamic size
- Efficient insertion/deletion
- Memory allocated as needed
- No memory waste

Disadvantages

- No random access
- Extra memory for pointers
- Sequential access only
- More complex implementation



Pointer-based List Structures III

Advantages

- Dynamic size
- Efficient insertion/deletion
- Memory allocated as needed
- No memory waste

Disadvantages

- No random access
- Extra memory for pointers
- Sequential access only
- More complex implementation



Study Case: Music Playlist Management with *Pointer-based List I*

Problem

Create a **music playlist** system where songs can be easily **added**, **removed**, and **reordered**. The playlist should handle an **unknown number** of songs efficiently.

Structure Song

title: String

artist: String

duration: Integer

next: Pointer to Song

end structure

Class Playlist

head: Pointer to Song

current: Pointer to Song

totalSongs: Integer



Study Case: Music Playlist Management with *Pointer-based List I*

Problem

Create a **music playlist** system where songs can be easily **added**, **removed**, and **reordered**. The playlist should handle an **unknown number** of songs efficiently.

Structure Song

title: String

artist: String

duration: Integer

next: Pointer to Song

end structure

Class Playlist

head: Pointer to Song

current: Pointer to Song

totalSongs: Integer



Study Case: Music Playlist Management with *Pointer-based List II*

Playlist Operations

- **Add Song:** Insert at **end** or specific position
- **Remove Song:** Delete by **title** or position
- **Play Next:** Move to **next song** in sequence
- **Shuffle:** Rearrange song order randomly
- **Search:** Find song by **title** or **artist**

Why *Linked List* Works Well

- Unknown number of songs
- Frequent additions/removals
- Sequential playback matches structure
- Easy to implement shuffle and repeat

Study Case: Music Playlist Management with *Pointer-based List II*

Playlist Operations

- **Add Song:** Insert at **end** or specific position
- **Remove Song:** Delete by **title** or position
- **Play Next:** Move to **next song** in sequence
- **Shuffle:** Rearrange song order randomly
- **Search:** Find song by **title** or **artist**

Why *Linked List* Works Well

- **Unknown number** of songs
- Frequent **additions/removals**
- **Sequential playback** matches structure
- Easy to implement **shuffle** and repeat

Thanks!

Questions?



Repo: <https://github.com/EngAndres/ud-public/tree/main/courses/computer-sciences-i>

